ChatLens Development Document

1. Product Name

ChatLens

2. Purpose

ChatLens is a WhatsApp QR-session based conversation intelligence system. Its main purpose is to read WhatsApp conversations, store them in a structured database, generate embeddings using PostgreSQL pgvector, and provide analytics, semantic search, customer insights, product demand detection, and conversation pattern analysis.

ChatLens is not primarily designed as a bulk messaging or campaign system. The first version should focus on message reading, storage, search, ML processing, and dashboards.

---

3. Technology Stack

Backend

- Django
- Django REST Framework
- PostgreSQL
- pgvector
- Celery
- Redis

WhatsApp QR Connector

- Node.js microservice
- Recommended library: Baileys or whatsapp-web.js
- Communicates with Django using internal REST API or queue

Frontend

- Django templates initially, or Vue.js if needed later
- Bootstrap / simple admin UI for first version

Database

- PostgreSQL as the main database
- pgvector extension for semantic embeddings

---

4. High-Level Architecture

```
WhatsApp Mobile
   ↓ scans QR
Node.js WhatsApp QR Worker
   ↓ captures messages
Django API
   ↓ stores normalized data
PostgreSQL
   ↓
Celery Processing Pipeline
   ↓
pgvector Embeddings
   ↓
Analytics + Semantic Search Dashboard
```

---

5. Core Django Apps

```
chatlens_core
- system settings
- users
- organization/account ownership

whatsapp_bridge
- WhatsApp accounts
- QR sessions
- chats
- contacts
- messages
- sync logs

message_intelligence
- embeddings
- semantic search
- intent detection
- product extraction
- sentiment analysis
- analytics
```

---

6. Main Features - Phase 1

6.1 QR Session Management

The system should allow a user to connect a WhatsApp account by scanning a QR code.

Required features:

```
- Create WhatsApp account record
- Generate QR code from Node.js worker
- Display QR code in Django UI
- Track session status
- Detect connected/disconnected/logged out state
- Reconnect existing session where possible
- Store session metadata
```

Session statuses:

```
pending_qr
qr_generated
connected
disconnected
logged_out
error
```

6.2 Message Ingestion

The Node.js worker should listen for incoming and outgoing WhatsApp messages and send them to Django.

Django should store:

```
- account
- chat
- contact
- message direction
- message text
- message type
- timestamp
- provider message id
- raw payload
```

Message types:

```
text
image
audio
video
document
sticker
location
contact
unknown
```

For Phase 1, media files do not need to be downloaded. Store only media metadata.

6.3 Contact and Chat Sync

ChatLens should maintain structured records for WhatsApp chats and contacts.

Required:

```
- individual chats
- group chats
- sender number
- display name
- chat name
- last message time
- unread count if available
- group participant metadata if available
```

6.4 Message Search

Implement normal keyword search first.

Search filters:

```
- account
- contact number
- chat
- date range
- direction
```

```
- message type
- keyword
```

6.5 pgvector Semantic Search

Enable semantic search over WhatsApp messages.

Example searches:

```
"customers asking for iPhone 17 Pro Max price"
"people complaining about warranty"
"customers asking for stock availability"
"payment follow-up messages"
"delivery delay complaints"
```

The system should return matching messages even when exact words are different.

---

7. Database Design

7.1 whatsapp_account

```sql
CREATE TABLE whatsapp_account (
    id BIGSERIAL PRIMARY KEY,
    owner_user_id BIGINT NOT NULL,
    display_name VARCHAR(255),
    phone_number VARCHAR(50),
    session_status VARCHAR(50) NOT NULL DEFAULT 'pending_qr',
    worker_session_id VARCHAR(255),
    last_connected_at TIMESTAMPTZ,
    last_disconnected_at TIMESTAMPTZ,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

7.2 whatsapp_contact

```sql
CREATE TABLE whatsapp_contact (
    id BIGSERIAL PRIMARY KEY,
    account_id BIGINT NOT NULL REFERENCES whatsapp_account(id) ON DELETE
CASCADE,
```

```
    wa_contact_id VARCHAR(255),
    phone_number VARCHAR(50),
    display_name VARCHAR(255),
    push_name VARCHAR(255),
    is_business BOOLEAN DEFAULT FALSE,
    raw_payload JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(account_id, wa_contact_id)
);
```

7.3 whatsapp_chat

```
CREATE TABLE whatsapp_chat (
    id BIGSERIAL PRIMARY KEY,
    account_id BIGINT NOT NULL REFERENCES whatsapp_account(id) ON DELETE
CASCADE,
    wa_chat_id VARCHAR(255) NOT NULL,
    chat_type VARCHAR(50) NOT NULL,
    name VARCHAR(255),
    contact_id BIGINT REFERENCES whatsapp_contact(id) ON DELETE SET NULL,
    last_message_at TIMESTAMPTZ,
    unread_count INTEGER DEFAULT 0,
    is_archived BOOLEAN DEFAULT FALSE,
    raw_payload JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(account_id, wa_chat_id)
);
```

7.4 whatsapp_message

```
CREATE TABLE whatsapp_message (
    id BIGSERIAL PRIMARY KEY,
    account_id BIGINT NOT NULL REFERENCES whatsapp_account(id) ON DELETE
CASCADE,
    chat_id BIGINT NOT NULL REFERENCES whatsapp_chat(id) ON DELETE CASCADE,
    contact_id BIGINT REFERENCES whatsapp_contact(id) ON DELETE SET NULL,
    provider_message_id VARCHAR(255) NOT NULL,
    sender_number VARCHAR(50),
    direction VARCHAR(20) NOT NULL,
    message_type VARCHAR(50) NOT NULL,
    message_text TEXT,
    message_time TIMESTAMPTZ NOT NULL,
    has_media BOOLEAN DEFAULT FALSE,
```

```
    media_mime_type VARCHAR(255),
    media_file_name VARCHAR(255),
    raw_payload JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE(account_id, provider_message_id)
);
```

7.5 message_embedding

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE message_embedding (
    id BIGSERIAL PRIMARY KEY,
    message_id BIGINT NOT NULL REFERENCES whatsapp_message(id) ON DELETE
CASCADE,
    embedding vector(1536),
    embedding_model VARCHAR(255) NOT NULL,
    metadata JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Index:

```
CREATE INDEX message_embedding_vector_idx
ON message_embedding
USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

7.6 message_analysis

```
CREATE TABLE message_analysis (
    id BIGSERIAL PRIMARY KEY,
    message_id BIGINT NOT NULL REFERENCES whatsapp_message(id) ON DELETE
CASCADE,
    language VARCHAR(20),
    sentiment VARCHAR(50),
    intent VARCHAR(100),
    urgency_score NUMERIC(5,2),
    lead_score NUMERIC(5,2),
    product_mentions JSONB,
    extracted_entities JSONB,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
    UNIQUE(message_id)
);
```

---

8. Django Model Structure

Recommended apps:

```
apps/
  chatlens_core/
    models.py
    permissions.py
    settings.py

  whatsapp_bridge/
    models/
      whatsapp_account.py
      whatsapp_contact.py
      whatsapp_chat.py
      whatsapp_message.py
      sync_log.py
    views.py
    serializers.py
    urls.py
    services/
      ingestion_service.py
      session_service.py

  message_intelligence/
    models/
      message_embedding.py
      message_analysis.py
    services/
      embedding_service.py
      semantic_search_service.py
      intent_service.py
      analytics_service.py
    tasks.py
    views.py
    serializers.py
    urls.py
```

---

9. Node.js QR Worker Responsibilities

The Node.js worker should only handle WhatsApp session connection and event capture.

Responsibilities:

```
- Create WhatsApp Web session
- Generate QR code
- Send QR code to Django
- Listen for connection status
- Listen for new messages
- Send message payloads to Django
- Handle reconnect
- Handle logout
```

The worker should expose internal endpoints:

```
POST /sessions
GET /sessions/:id/qr
POST /sessions/:id/disconnect
GET /sessions/:id/status
```

The worker should call Django endpoints:

```
POST /api/internal/whatsapp/session-status/
POST /api/internal/whatsapp/message-ingest/
```

---

10. Django Internal API Endpoints

Session Status

```
POST /api/internal/whatsapp/session-status/
```

Payload:

```
{
  "worker_session_id": "session_123",
  "status": "connected",
  "phone_number": "9715xxxxxxx",
  "display_name": "Business Name",
```

```
  "event_time": "2026-06-17T10:00:00Z"
}
```

Message Ingest

```
POST /api/internal/whatsapp/message-ingest/
```

Payload:

```
{
  "worker_session_id": "session_123",
  "provider_message_id": "ABC123",
  "chat_id": "9715xxxxxxx@s.whatsapp.net",
  "chat_type": "individual",
  "sender_number": "9715xxxxxxx",
  "direction": "inbound",
  "message_type": "text",
  "message_text": "iPhone 17 Pro Max orange available?",
  "message_time": "2026-06-17T10:01:00Z",
  "raw_payload": {}
}
```

---

11. Message Processing Pipeline

When a message is received:

```
1. Validate internal API token
2. Identify WhatsApp account by worker_session_id
3. Upsert contact
4. Upsert chat
5. Insert message
6. Queue Celery task for analysis
7. Generate embedding
8. Store vector
9. Run intent/entity extraction
10. Update analytics tables
```

Celery tasks:

```
process_message_analysis(message_id)
generate_message_embedding(message_id)
```

```
extract_product_mentions(message_id)
refresh_daily_analytics(account_id, date)
```

---

12. Embedding Strategy

For Phase 1:

```
One embedding per text message
Skip empty messages
Skip unsupported media messages unless caption exists
Store embedding model name
Store metadata with account_id, chat_id, direction, message_time, intent
```

Metadata example:

```
{
  "account_id": 1,
  "chat_id": 55,
  "contact_id": 19,
  "direction": "inbound",
  "message_time": "2026-06-17T10:01:00Z",
  "intent": "stock_inquiry",
  "products": ["iPhone 17 Pro Max", "Orange"]
}
```

---

13. Semantic Search Flow

User query:

```
"customers asking for iPhone 17 Pro Max orange"
```

System flow:

```
1. Generate embedding for query
2. Search message_embedding using cosine distance
3. Join result with whatsapp_message
4. Apply filters
5. Return top matches
```

SQL example:

```
SELECT
    wm.id,
    wm.message_text,
    wm.message_time,
    wm.sender_number,
    me.embedding <=> %(query_embedding)s AS distance
FROM message_embedding me
JOIN whatsapp_message wm ON wm.id = me.message_id
WHERE wm.account_id = %(account_id)s
ORDER BY me.embedding <=> %(query_embedding)s
LIMIT 30;
```

---

14. Analytics Dashboard

Phase 1 dashboard cards:

```
Total messages today
Inbound messages today
Outbound messages today
Top active chats
Average response time
Unanswered customer messages
Top product mentions
Top customer intents
Complaint messages
Hot leads
```

Charts:

```
Messages by day
Messages by hour
Top requested products
Intent distribution
Sentiment distribution
Response-time trend
```

---

15. ML / Intelligence Features

Initial intent categories:

```
price_inquiry
stock_inquiry
order_request
invoice_request
payment_followup
delivery_followup
warranty_complaint
general_complaint
greeting
unknown
```

Initial product extraction should use:

```
- keyword dictionary
- fuzzy matching
- known product list from ERP if available later
- vector similarity for alternate spellings
```

Examples:

```
17pm
17 promax
iPhone 17 max
17 pro max 256
```

Should all map close to:

```
iPhone 17 Pro Max
```

---

16. Read-First Controls

Initial version should keep sending functionality disabled.

System settings:

```
send_message_enabled = false
auto_reply_enabled = false
bulk_send_enabled = false
```

The UI should not expose message sending in Phase 1.

---

17. Permissions

Suggested permissions:

```
chatlens.view_dashboard
chatlens.view_accounts
chatlens.manage_accounts
chatlens.view_chats
chatlens.view_messages
chatlens.view_analytics
chatlens.run_semantic_search
chatlens.export_data
chatlens.manage_sessions
```

---

18. Admin Screens

Required screens:

```
Dashboard
WhatsApp Accounts
QR Connect Screen
Session Status
Chats
Contacts
Messages
Semantic Search
Analytics
Processing Logs
Settings
```

---

19. Development Phases

Phase 1 - Foundation

```
- Django project setup
- PostgreSQL setup
- pgvector extension
- base apps
- WhatsApp account model
- contact/chat/message models
- internal ingestion API
- basic admin UI
```

Phase 2 - QR Worker

```
- Node.js worker setup
- QR generation
- session connect/disconnect
- message listener
- send message events to Django
- session status updates
```

Phase 3 - Message Intelligence

```
- Celery + Redis
- embedding generation
- pgvector storage
- semantic search endpoint
- message analysis model
- basic intent detection
```

Phase 4 - Dashboard

```
- message volume dashboard
- top chats
- top product mentions
- intent distribution
- missed inquiry detection
- response-time analytics
```

Phase 5 - ERP Integration

```
- connect product master from Cellusense
- map product mentions to inventory products
- detect demand by product
- generate sales lead suggestions
- optional CRM prospect creation
```

---

20. First Development Instruction

Start by creating the Django project named:

```
chatlens
```

Create the following apps:

```
chatlens_core
whatsapp_bridge
message_intelligence
```

Configure:

```
PostgreSQL
pgvector
Django REST Framework
Celery
Redis
```

Then implement the database models for:

```
WhatsAppAccount
WhatsAppContact
WhatsAppChat
WhatsAppMessage
MessageEmbedding
MessageAnalysis
```

After models are complete, implement the internal message ingestion API:

```
POST /api/internal/whatsapp/message-ingest/
POST /api/internal/whatsapp/session-status/
```

Do not implement sending features in the first phase.

Focus first on:

```
QR session status
message ingestion
message storage
message analysis queue
pgvector preparation
```